In [ ]:
```
# Initialize Otter
import otter
grader = otter.Notebook("practice06.ipynb")
```

Student names and e-mails:

YOUR NAME — your@calvin.edu

YOUR NAME — your@calvin.edu

Practice 06 — Who Lives Here? Decision Trees, Cross-Validation and Grid Search

Field researchers record what kind of organism they find at a site, along with the site's elevation, temperature and rainfall. Can a few yes/no questions about a site tell us what lives there? How many questions is too many? And how do we choose without peeking at the test set?

SLO	Description
07A	Train and interpret decision tree models and explain how tree depth affects overfitting and underfitting
07B	Use cross-validation to evaluate model performance across multiple data splits
07C	Tune model hyperparameters using grid search

The Dataset: Organisms and Sites



The data is **simulated**: 1,000 observations, each one organism found at one site. Being simulated, its patterns were put there on purpose, and at the end of the practice you'll see exactly how. That makes it a good place to practice, but nothing here is a claim about real ecology.

Like real field data, it is messy: some sightings are out of place (a bird that wandered far from home, a misidentified insect), so no model can get every site right.

Column	What it holds
`elevation_m`	the site's elevation, in meters
`temperature_c`	the site's average temperature, in °C
`precipitation_mm`	the site's yearly rainfall, in mm
`soil`	soil type: Sandy, Clay, Loamy, Peaty or Chalky
`habitat`	habitat type: Forest, Grassland, Wetland or Desert
`organism`	**the label**: Plant, Insect, Bird, Mammal or Reptile

Setup (given — just run it)

In [ ]:
```
import numpy as np
import pandas as pd
import plotly.express as px
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score

sites = pd.read_csv("https://cs.calvin.edu/courses/data/202/26fa/datasets/species.csv")
print(len(sites), "observations")
print(sites["organism"].value_counts().to_string())
sites.head()
```

Two columns we leave out. `soil` and `habitat` are words, not numbers, and scikit-learn's trees only accept numbers. Turning categories into numbers is a topic for later in the course. Here we lose little by dropping them. `habitat` is just rainfall in four bands, which `precipitation_mm` already tells us in more detail:

In [ ]:
```
sites.groupby("habitat")["precipitation_mm"].agg(["min", "max"])
```

And in this data, `soil` has nothing to do with which organism shows up. So the features are the three numeric columns, and the label is `organism`. The split is given: 20% for testing, with `stratify=y` so both sets keep the same share of each organism. **The test set stays locked** until a step says to use it.

In [ ]:
```
features = ["elevation_m", "temperature_c", "precipitation_mm"]
X = sites[features]
y = sites["organism"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, ran
dom_state=42)
print(len(X_train), "training sites |", len(X_test), "test sites")
```

Part 1 — A Tree With Two Questions (SLO 07A)

> 🎯 **Predict First:** A tree with `max_depth=2` asks at most 2 questions per site. How many final boxes (leaves) can it have? There are 5 organisms. Can it predict all of them?

Task 07A.1 — Fit a Depth-2 Tree *(2 pts)*

Fit a `DecisionTreeClassifier` with `max_depth=2` and `random_state=0` on the **training** set, and store it in `tree2`. Then store its accuracy on the training set in `train_acc` and on the test set in `test_acc`.

The plot of the tree is given.

In [ ]:
```
tree2 = DecisionTreeClassifier(max_depth=2, random_state=0).fit(X_train, y_train)  # SOLUTION
train_acc = accuracy_score(y_train, tree2.predict(X_train))  # SOLUTION
test_acc = accuracy_score(y_test, tree2.predict(X_test))  # SOLUTION
print(f"training accuracy: {train_acc:.3f} | test accuracy: {test_acc:.3f}")

plt.figure(figsize=(14, 6))
plot_tree(tree2, feature_names=features, class_names=tree2.classes_, filled=True, fontsize=9)
plt.show()
```

In [ ]:
```
grader.check("07A.1")
```

Does the Tree Make Sense? (given — just run it)

The tree first asks about `precipitation_mm`, then about `temperature_c` on the drier side. Here are the training sites on those two features, with those two cuts drawn in:

In [ ]:
```
fig = px.scatter(X_train.assign(organism=y_train), x="precipitation_mm", y="temperature_c", color="organism",
                 opacity=0.7, title="800 Training Sites: Rainfall vs. Temperature")
fig.add_vline(x=1774.65, line_dash="dash", line_color="black")               # the root question
fig.add_shape(type="line", x0=0, x1=1774.65, y0=19.65, y1=19.65,            # the second question, drier side only
              line=dict(dash="dot", color="black"))
fig.show()
```

Question 07A.2 ***(1 pt)*****.** Read the tree and the plot.

1. In words, what does the tree predict for a **drier** site (precipitation ≤ 1774.65 mm) that is **warm** (temperature > 19.65 °C)? How sure is it — look at `value` in that leaf?
2. Which organism does the tree **never** predict? Look at where those organisms sit in the `value` lists of the four leaves, and where they sit in the plot. Why can't a depth-2 tree predict it?

Type your answer here, replacing this text.

Sample answer:

1. Drier and warm → **Insect**. But it's not very sure: that leaf holds 313 training sites, and `value` = [25, 159, 17, 16, 96], so only 159 (51%) are insects, and 96 are reptiles.
2. It never predicts **Reptile**. A depth-2 tree has at most 4 leaves, but there are 5 organisms, so at least one can't win any leaf. The reptiles are in the warm, drier leaf, outnumbered by insects (96 to 159). In the plot they sit at the far left: the *driest* sites. Separating them needs a third question (is the rainfall very low?), so the tree is **too simple** for this data. That's underfitting.

Part 2 — How Many Questions? Depth and Overfitting (SLO 07A)

Depth 2 gets 63% of the test sites right. That's far better than guessing one of 5 (about 20%), but it can't even predict reptiles. More questions should help. But how many?

Task 07A.3 — Training vs. Test, for Every Depth *(3 pts)*

For every depth in `depths` (1 to 22; at depth 22 every leaf is pure, so the tree can't grow further), fit a tree with that `max_depth` and `random_state=0` on the training set. Store its accuracy on the training set and on the test set, and also the number of leaves (`model.get_n_leaves()`). The loop collects them into `scores` ; the plot is given.

(We look at the test set here only to see the overfitting shape, like the decision-trees class before Fall Break. In Part 3 we'll choose the depth the fair way.)

In [ ]:
```
depths = list(range(1, 23))
rows = []
for depth in depths:
    # fit a tree with this max_depth on the training set
    model = DecisionTreeClassifier(max_depth=depth, random_state=0).fit(X_train, y_trai
n)  # SOLUTION
    # accuracy on the training set and on the test set
    acc_train = accuracy_score(y_train, model.predict(X_train))  # SOLUTION
    acc_test = accuracy_score(y_test, model.predict(X_test))  # SOLUTION
    rows.append({"max_depth": depth, "leaves": model.get_n_leaves(), "train": acc_train,
"test": acc_test})
scores = pd.DataFrame(rows)

px.line(scores, x="max_depth", y=["train", "test"], markers=True,
        labels={"value": "accuracy", "variable": "scored on"},
        title="Decision Tree Accuracy vs. Depth").show()
scores.round(3)
```

In [ ]:
```
grader.check("07A.3")
```

Question 07A.4 ***(1 pt)*****.** Use the plot and the table.

1. Where is the tree **underfitting**? Where is it **overfitting**? What in the plot tells you?
2. The deepest tree has 228 leaves for 800 training sites and scores 100% on them. What are all those leaves learning, and what happens to them on the test sites? *(Remember: some sightings are out of place.)*

Type your answer here, replacing this text.

Sample answer:

1. **Underfitting** at depths 1–2 (and a bit at 3): training and test accuracy are both low (40% and 63%). The tree asks too few questions to find the organisms' regions. The best test score is around **depth 4** (80.5%). **Overfitting** from about depth 5 on: training accuracy keeps climbing to 100%, while test accuracy *falls*, down to about 66%. The growing **gap** between the two lines is the sign.
2. They are memorizing individual sites: 167 of the 228 leaves hold just one or two training sites (139 hold a single site). Many of those are the out-of-place sightings, a bird in a lowland rainforest, say, and the tree draws a tiny box around each one. Those boxes don't describe any real pattern, so on new sites they just make mistakes: a new site that lands in the stray bird's box is called a bird. That's why test accuracy drops from 80.5% to 65.5%. (And all of these test scores come from **one split** of 200 sites.)

Part 3 — Choosing Fairly: Cross-Validation (SLO 07B)

In Part 2 the test set did the judging. If we pick the depth that way, the test set is no longer "new data". The fix from the last class before Fall Break: keep the test set locked, and judge inside the **training** set with **5-fold cross-validation**:

1. Cut the 800 training sites into 5 folds of 160
2. Fit on 4 folds, score on the 5th. Repeat 5 times so every fold takes a turn
3. The mean of the 5 scores is the cross-validation score

Task 07B.1 — Cross-Validating the Full Tree *(2 pts)*

The tree with **no depth limit** scored 100% on the training set. Use `cross_val_score` with `cv=5` to score a `DecisionTreeClassifier(random_state=0)` (no `max_depth`) on the **training** set. Store the 5 scores in `fold_scores` and their mean in `cv_mean`.

In [ ]:
```
fold_scores = cross_val_score(DecisionTreeClassifier(random_state=0), X_train, y_train,
cv=5)  # SOLUTION
cv_mean = fold_scores.mean()  # SOLUTION
print("5 rounds:", fold_scores.round(3))
print("mean:    ", round(cv_mean, 3))
```

In [ ]:
```
grader.check("07B.1")
```

100% on the data it learned from; about 63% on the folds it didn't see. Cross-validation tells the same story as the test set — without touching it.

Task 07B.2 — A Cross-Validation Score for Every Depth *(3 pts)*

For every depth in `depths`, compute the 5-fold cross-validation scores of a tree with that `max_depth` (and `random_state=0`) on the **training** set. Store the **mean** and the **standard deviation** (`.std()`, how much the 5 rounds disagree) of the 5 scores. The plot is given.

Then store in `best_depth` the depth with the highest mean cross-validation score. *(Hint: `cv_scores.loc[cv_scores["mean"].idxmax(), "max_depth"]`.)*

In [ ]:
```
rows = []
for depth in depths:
    # 5-fold cross-validation scores for a tree with this max_depth, on the training set
    s = cross_val_score(DecisionTreeClassifier(max_depth=depth, random_state=0), X_train, y_train, cv=5)  # SOLUTION
    rows.append({"max_depth": depth, "mean": s.mean(), "spread": s.std()})
cv_scores = pd.DataFrame(rows)

best_depth = cv_scores.loc[cv_scores["mean"].idxmax(), "max_depth"]  # SOLUTION
print("best depth by cross-validation:", best_depth)

px.line(cv_scores, x="max_depth", y="mean", error_y="spread", markers=True,
        labels={"mean": "mean CV accuracy"},
        title="5-Fold Cross-Validation Accuracy vs. Depth (800 training sites)").show()
```

In [ ]:
```
grader.check("07B.2")
```

Now, **once**, open the locked test set for the depth that cross-validation chose:

In [ ]:
```
final_tree = DecisionTreeClassifier(max_depth=best_depth, random_state=0).fit(X_train, y_train)   # all 800 training sites
print(f"depth {best_depth} | CV: {cv_scores['mean'].max():.3f} | test: {accuracy_score(y_test, final_tree.predict(X_test)):.3f}")
```

> 📌 **One split:** that test score is on *these* 200 test sites.

Part 4 — Tuning Several Knobs: Grid Search (SLO 07C)

`max_depth` isn't the only knob. The pruning knobs from Week 7 also include `min_samples_leaf` (every leaf needs at least that many sites) and `min_samples_split` (a box needs at least that many sites before it may ask another question). **Grid search** cross-validates every combination and keeps the best one.

Task 07C.1 — Grid Search *(3 pts)*

1. Build a `GridSearchCV` with a `DecisionTreeClassifier(random_state=0)`, the grid `grid` below, and `cv=5`. Store it in `search` and fit it on the **training** set.
2. Store how many trees the search fit during cross-validation in `n_fits` — combinations × folds, as a number. *(Count the combinations from `grid`; don't count the final refit.)*
3. Store the best cross-validation accuracy in `best_cv`, and the accuracy of the winning tree (`search.best_estimator_`) on the **test** set in `winner_test`.

The table of the top 10 combinations is given.

In [ ]:
```
grid = {"max_depth": [2, 4, 6, 8, 10, None],          # None = no limit
        "min_samples_leaf": [1, 5, 10, 20],
        "min_samples_split": [2, 10, 20]}

# build the grid search, then fit it on the training set
search = GridSearchCV(DecisionTreeClassifier(random_state=0), grid, cv=5)  # SOLUTION
search.fit(X_train, y_train)  # SOLUTION
# trees fit during cross-validation: combinations x folds
n_fits = 6 * 4 * 3 * 5  # SOLUTION

# best CV accuracy, and the winner's accuracy on the test set
best_cv = search.best_score_  # SOLUTION
winner_test = accuracy_score(y_test, search.best_estimator_.predict(X_test))  # SOLUTION
print("best combination:", search.best_params_)
print(f"best CV accuracy: {best_cv:.3f} | test accuracy: {winner_test:.3f} | trees fit:
{n_fits}")

results = pd.DataFrame(search.cv_results_)
results[["param_max_depth", "param_min_samples_leaf", "param_min_samples_split", "mean_t
est_score", "std_test_score"]] \
    .sort_values("mean_test_score", ascending=False).head(10).round(3)
```

In [ ]:
```
grader.check("07C.1")
```

Three Numbers for Two Trees (given — just run it)

In [ ]:
```
full = DecisionTreeClassifier(random_state=0).fit(X_train, y_train)
winner = search.best_estimator_
pd.DataFrame({
    "leaves": [full.get_n_leaves(), winner.get_n_leaves()],
    "training accuracy": [accuracy_score(y_train, full.predict(X_train)), accuracy_score
(y_train, winner.predict(X_train))],
    "CV accuracy": [cv_mean, best_cv],
    "test accuracy": [accuracy_score(y_test, full.predict(X_test)), winner_test],
}, index=["no limit", "grid winner"]).round(3)
```

Question 07C.2 *(1 pt)*. Look at the table above and the top-10 table.

1. Did the grid winner overfit? Compare its training, CV and test accuracy, and compare it with the tree with no limit.
2. Look at the top-10 table. Several combinations tie for first place, and the plain depth-4 tree (`min_samples_leaf=1` , `min_samples_split=2`) is close behind. How many of the 800 training sites does that difference amount to? Which tree would you choose, and why?

Type your answer here, replacing this text.

Sample answer:

1. No. The winner (depth 4, 15 leaves) scores 80% on the training sites, 78.5% in cross-validation and 80.5% on the test sites: no real gap. The tree with no limit scores 100% on training but only 63% in CV and 65.5% on test, with 228 leaves. That gap is overfitting, and the grid search steered away from it.
2. Four combinations tie at 0.785; the plain depth-4 tree scores 0.781. That's 0.004 × 800 ≈ **3 sites**. Nine of the top ten have `max_depth=4` ; the leaf and split settings barely matter. With 72 combinations, which one lands exactly on top is partly the luck of these 5 folds. The best of many is optimistic. The sensible choice is the simplest model that does about as well: `max_depth=4` , perhaps with a small `min_samples_leaf` for safety. And 80.5% is from **one split** of 200 test sites.

The Answer Key: How the Data Was Made (given — just run it)

With real data, nobody hands you the true rules. Here we can peek, because we simulated the data. Each site's organism came from these rules:

if…	organism
rainfall < 600 mm and temperature > 18 °C	Reptile
rainfall < 600 mm and temperature ≤ 18 °C	Mammal
otherwise, elevation > 1500 m	Bird
otherwise, rainfall > 1800 mm	Plant
otherwise, temperature > 20 °C	Insect
otherwise	Mammal

Then **1 sighting in 4** was replaced by a random organism: the out-of-place sightings. No model can predict those, so even the true rules get only about **80%** right:

In [ ]:
```
E, T, P = X_test["elevation_m"], X_test["temperature_c"], X_test["precipitation_mm"]
true_rules = np.where(P < 600, np.where(T > 18, "Reptile", "Mammal"),
             np.where(E > 1500, "Bird",
             np.where(P > 1800, "Plant",
             np.where(T > 20, "Insect", "Mammal"))))
print("the true rules, on the test sites:", accuracy_score(y_test, true_rules))
```

- Compare the rules with your depth-4 tree's questions (`plot_tree(search.best_estimator_, ...)` if you're curious): rainfall ≈ 1775 and ≈ 600, temperature ≈ 19.7, elevation ≈ 1485. The tree found almost the same cuts from 800 noisy sites
- Depth 4 scores about as well as the **true rules**. Every question past that point can only learn the out-of-place sightings, which is exactly what overfitting means
- Depths 1–2 can't even express the rules: they have too few leaves. That's underfitting
- In real data, there is no answer key, so we can't know where the 80% ceiling is. That's why we let cross-validation choose

---

Submission

Save your notebook, then upload the **`.ipynb` file** directly to our Moodle assignment page. Don't export, zip, or submit anything else — just the notebook.